

Angular Moderno

19 - Tratamento de Erros



Angular Moderno

19 - Tratamento de Erros:

Estratégias Reais e UX em Falhas

Continuidade direta das aulas anteriores, evoluindo o mesmo projeto de e-commerce com lista de produtos, agora com foco em experiência do usuário em situações de erro.

Objetivo da Aula

O que já temos

- Requisição HTTP funcionando
- Interceptor global com logs
- Tratamento técnico de erros no interceptor

Novo objetivo desta aula

- Implementar tratamento de erro voltado ao **usuário**
- Melhorar a experiência em caso de falha
- Criar estados de UI explícitos e controlados

 O foco agora é a **UX de erro no componente**: o que o usuário vê quando algo dá errado.



O Problema Atual

Tela vazia

Sem feedback visual, o usuário vê uma tela em branco sem entender o motivo.

Sem contexto

O usuário não sabe se está carregando, se houve erro ou se não há dados.

Sistema quebrado

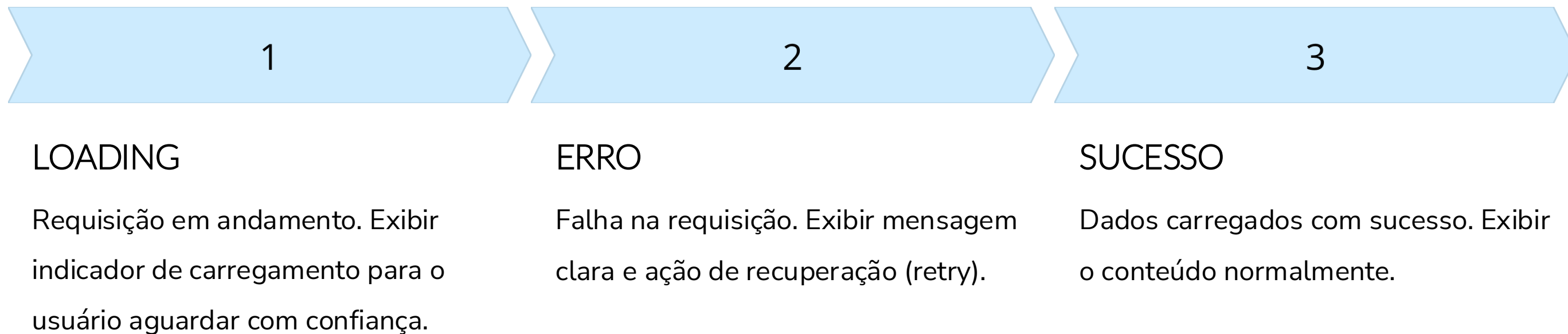
A sensação transmitida é de que o sistema está com defeito, causando uma péssima experiência para o usuário.

⚠ No código atual, quando ocorre erro, o bloco `@if (!carregando())` ainda é executado.

❌ **Erro técnico ≠ Experiência do usuário.** O interceptor trata o erro internamente, mas o usuário ainda não vê nada útil na tela.

Conceito Central: Os 3 Estados da UI

Toda interface que faz requisições assíncronas precisa controlar **explicitamente** três estados distintos. Ignorar qualquer um deles resulta em UX ruim.



📌 Esses estados devem ser **mutuamente exclusivos** — apenas um deve estar ativo por vez.

Situação Atual do Projeto

O que o componente trata hoje

carregando()

Exibe indicador enquanto a requisição está em andamento.

Dados carregados

Exibe a lista de produtos quando a requisição tem sucesso.

O problema

O estado de **ERRO** não é tratado, fazendo com que a UI entre em um estado inconsistente.

Resultado: **tela vazia, sem mensagem, sem ação possível para o usuário.**



Etapa 0: Parar o Servidor



Antes de qualquer alteração no código, pare o servidor de desenvolvimento!

Como parar

No terminal onde o servidor está rodando, pressione:

CTRL + C

Por que parar?

Evita inconsistências durante a edição dos arquivos e garante que as alterações sejam aplicadas corretamente ao reiniciar.

Boa prática

Profissionais sempre param o servidor antes de alterações estruturais no projeto. Adote esse hábito desde o início.

Etapa 1: Criar o Estado de Erro

LISTA-PRODUTOS.TS

O primeiro passo é criar um **Signal** dedicado para armazenar o estado de erro. Esse signal vai controlar se há ou não uma mensagem de erro para exibir ao usuário.

Adicionar **dentro da classe** do componente **lista-produtos**:

```
erro = signal<string | null>(null);
```

Valor **null**

Sem erro. Estado padrão da aplicação. Nenhuma mensagem de erro é exibida.

Valor **string**

Há um erro. A string contém a mensagem que será exibida ao usuário na interface.

 Usamos **Signals** porque eles são reativos — quando o valor muda, o template é atualizado automaticamente.

Isso é Angular Moderno!

Etapa 2: Limpar o Erro Antes da Requisição

LISTA-PRODUTOS.TS

Dentro do método `carregarProdutos()`, antes de iniciar a requisição, é necessário limpar qualquer erro anterior e ativar o estado de carregamento.

```
carregarProdutos() {  
  this.erro.set(null); // limpa erro anterior  
  this.carregando.set(true); // ativa loading  
  this.produtosService.buscarProdutos().subscribe({  
    next: (dados) => {  
      const produtos = this.produtosService.transformarProdutos(dados);  
      this.produtos.set(produtos);  
      this.carregando.set(false);  
    },  
    error: (erro) => {  
      console.error('Erro ao carregar produtos:', erro);  
      this.erro.set('Erro ao carregar produtos. Verifique sua conexão e tente novamente.');
```

1 Evitar erro antigo

Se o usuário clicar em "Tentar novamente", o erro anterior deve desaparecer imediatamente — não esperar a resposta da nova requisição.

2 Garantir consistência

A UI deve refletir o estado atual da operação. Limpar o erro garante que loading e erro não apareçam ao mesmo tempo.

Etapa 3: Tratar o Erro no Subscribe

LISTA-PRODUTOS.TS

No bloco `subscribe()`, adicione o callback de `error` para capturar falhas e atualizar os signals corretamente.

```
carregarProdutos() {  
  this.erro.set(null); // limpa erro anterior  
  this.carregando.set(true); // ativa loading  
  this.produtosService.buscarProdutos().subscribe({  
    next: (dados) => {  
      const produtos = this.produtosService.transformarProdutos(dados);  
      this.produtos.set(produtos);  
      this.carregando.set(false);  
    },  
    error: (erro) => {  
      console.error('Erro ao carregar produtos:', erro);  
      this.erro.set('Erro ao carregar produtos. Verifique sua conexão e tente novamente.');
```

`console.error` → Técnico

O log no console é para o **desenvolvedor**. Contém detalhes técnicos do erro para debugging.

signal erro → Visual

A mensagem no signal é para o **usuário**. Clara, amigável e com orientação de ação.

Etapa 4: Ajustar o Template

LISTA-PRODUTOS.HTML

⊗ **PONTO CRÍTICO** — **Leia com atenção!** O bloco antigo DEVE SER SUBSTITUÍDO, não apenas complementado.
Adicionar código novo sem remover o bloco antigo gera conflito de estados!

Bloco ANTIGO — Remover

```
@if (!carregando()) {  
  <!-- lista de produtos -->  
}
```

Este bloco não distingue entre erro e sucesso.
Deve ser completamente substituído.

Estrutura NOVA — Inserir (Completo no próx slide)

```
@if (carregando()) { ... }  
@if (erro()) { ... }  
@if (!carregando() && !erro()) { ... }
```

Três blocos independentes, cada um responsável por um estado da UI.

Etapa 4 (Continuação): Inserir o Bloco de Erro

LISTA-PRODUTOS.HTML

Adicione o bloco de erro no template. Ele exibe a mensagem ao usuário e oferece uma ação de recuperação.

```
@if (carregando()) {  
  <p>Carregando produtos...</p>  
}  
  
@if (erro()) {  
  <p style="color: red">  
    {{ erro() }}  
  </p>  
  <button (click)="carregarProdutos()">Tentar novamente</button>  
}  
  
@if (!carregando() && !erro()) {  
  <p>Total de produtos: {{ totalProdutos() }}</p>  
  <hr />  
  @for (item of produtos(); track $index) {  
    <app-produto [nome]="item.nome" [preco]="item.preco" />  
  }  
}
```



Exibição da mensagem

O `{{ erro() }}` exibe o valor atual do signal de erro (a mensagem amigável definida no componente).



Ação de retry

O botão chama `carregarProdutos()` novamente, permitindo que o usuário tente recuperar sem recarregar a página.

Etapa 4: Fluxo Completo do Template

LISTA-PRODUTOS.HTML

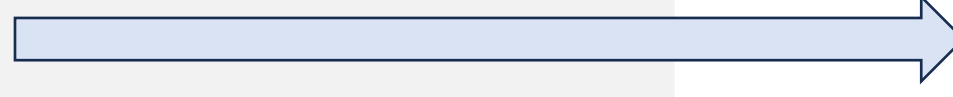
A estrutura final do template com os três estados mutuamente exclusivos:

```
<!-- Estado 1: Loading -->
@if (carregando()) {
  <p>Carregando produtos...</p>
}
```



carregando() = true
Apenas o loading aparece

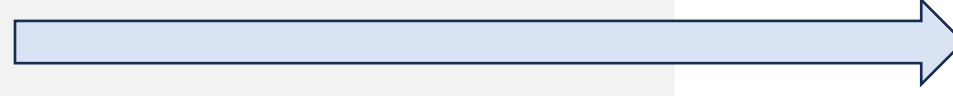
```
<!-- Estado 2: Erro -->
@if (erro()) {
  <p style="color: red">
    {{ erro() }}
  </p>
}
```



erro() ≠ null
Apenas o erro aparece

```
<button (click)="carregarProdutos()">Tentar novamente</button>
}
```

```
<!-- Estado 3: Sucesso -->
@if (!carregando() && !erro()) {
  <p>Total de produtos: {{ totalProdutos() }}</p>
}
```



Ambos falsos
Apenas os produtos aparecem

```
<hr />
```

```
@for (item of produtos(); track $index) {
  <app-produto [nome]="item.nome" [preco]="item.preco" />
}
}
```

Integração com o Projeto Atual

É importante entender que as alterações desta aula são **cirúrgicas** — afetam apenas o fluxo de carregamento de produtos, sem impactar outras funcionalidades.

produtoSelecionado — sem alteração

A lógica de seleção de produto continua exatamente igual. Nenhuma linha foi tocada.

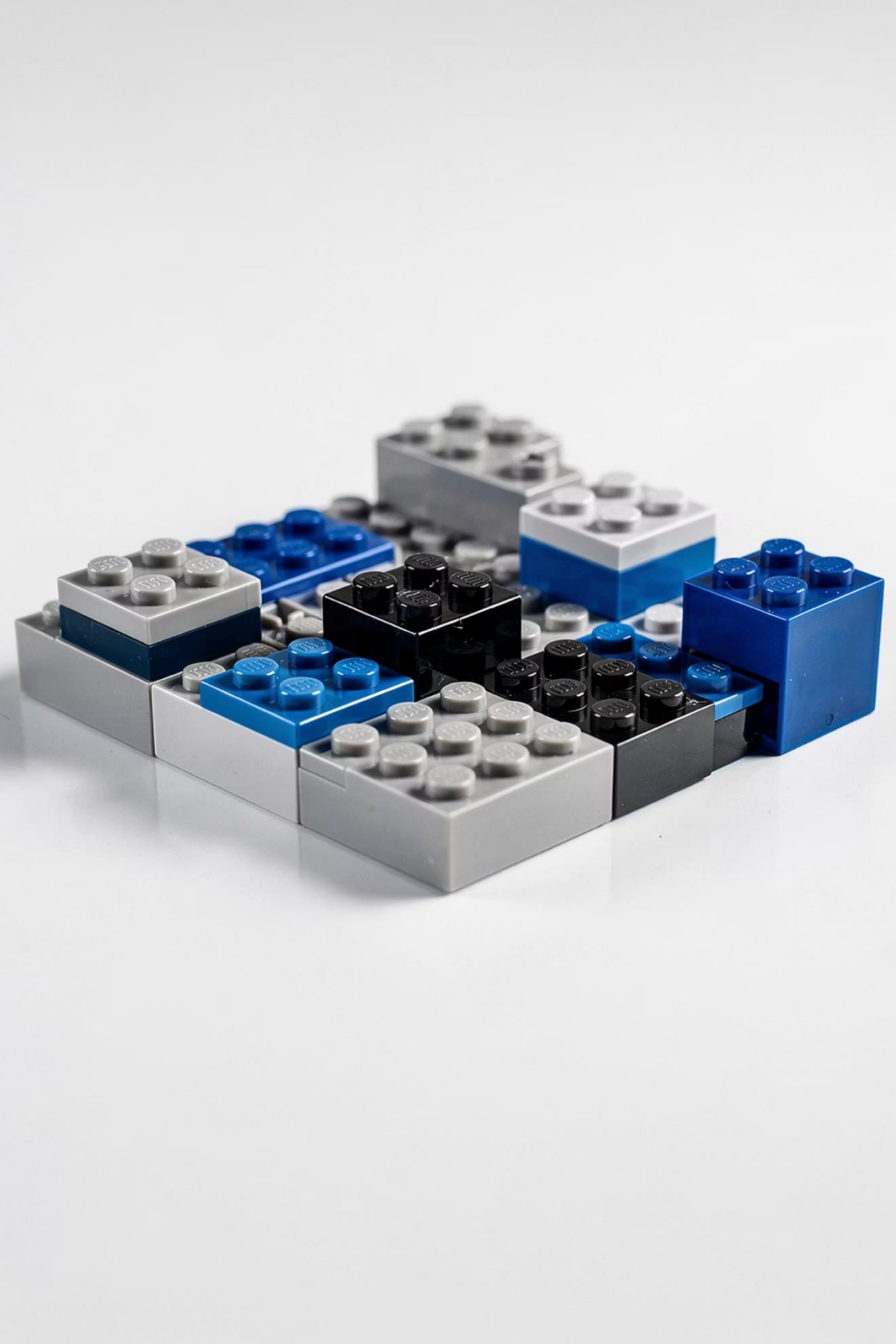
Carrinho — sem alteração

O serviço de carrinho e toda sua lógica permanecem intactos e funcionando normalmente.

Fluxo de produtos — ajustado

Apenas o subscribe de carregamento de produtos recebeu o novo callback de erro e os signals foram atualizados.

✔ **Separação de responsabilidades em ação!** Cada parte do sistema cuida do seu próprio domínio. Isso é arquitetura de qualidade.



Etapa 5: Simular um Erro

PRODUTOS.SERVICE.TS

Para testar se o tratamento de erro está funcionando corretamente, vamos simular uma falha alterando a URL da API para um endpoint inválido.

```
// Alterar temporariamente para URL inválida  
private API = 'https://fakestoreapi.com/products-erro';
```

Por que simular?

Em desenvolvimento, não podemos depender de falhas reais de rede. Simular erros de forma controlada é uma prática profissional essencial para validar o comportamento da UI.

O que esperar

- A requisição retornará erro 404
- O interceptor registrará o erro no console
- O componente exibirá a mensagem de erro
- O botão "Tentar novamente" aparecerá

 Lembre-se: esta alteração é **temporária**. Será revertida na próxima etapa.

Resultado do Teste de Erro

(0) Minha Loja

⌵

⌵

⌵

←

→

↻

🏠

🔒 http://localhost:4200/produtos

📄

🔍

☆

😊

📄

☰

Minha Loja

[Home](#) | [Produtos](#) | [Carrinho](#)

Login

Logout

Status: Deslogado

Produtos:

Erro ao carregar produtos. Verifique sua conexão e tente novamente.

Tentar novamente

🔍

🔍 Inspetor

📄 Console

🔍 Debugger

↕ Rede

📄 Editor de estilos

🔍 Desempenho

📄 Memória

📄 Armazenamento

👤 Acessibilidade

📄 Aplicação

🔴 4

📄

⌵

🗑️

🔍 Filtrar saída

Erros

Warnings

Info

Logs

Debug

CSS

XHR

Requisições

⚙️

[vite] connecting...

[vite] connected.

Angular is running in development mode.

Angular hydrated 2 component(s) and 27 node(s), 0 component(s) were skipped. Learn more at <https://v21.angular.dev/guide/hydration>.

REQUEST <https://fakestoreapi.com/products-erro>

RESPONSE: ▶ `Object { type: 0 }`

Lista de produtos alterada: ▶ `Array []`

Valor total atualizado: 0

▶ **XHR** GET <https://fakestoreapi.com/products-erro> [HTTP/2 404 311ms]

❗ ▶ **ERRO:**
▶ `Object { headers: {...}, status: 404, statusText: "Unknown Error", url: "https://fakestoreapi.com/products-erro", ok: false, type: undefined, redirected: false, responseType: "cors", name: "HttpErrorResponse", message: "Http failure response for https://fakestoreapi.com/products-erro: 404 ", ... }`

❗ ▶ **ERRO GLOBAL:**
▶ `Object { headers: {...}, status: 404, statusText: "Unknown Error", url: "https://fakestoreapi.com/products-erro", ok: false, type: undefined, redirected: false, responseType: "cors", name: "HttpErrorResponse", message: "Http failure response for https://fakestoreapi.com/products-erro: 404 ", ... }`

❗ ▶ **Erro ao carregar produtos:**
▶ `Object { headers: {...}, status: 404, statusText: "Unknown Error", url: "https://fakestoreapi.com/products-erro", ok: false, type: undefined, redirected: false, responseType: "cors", name: "HttpErrorResponse", message: "Http failure response for https://fakestoreapi.com/products-erro: 404 ", ... }`

⏏

Etapa 6: Restaurar a API

PRODUTOS.SERVICE.TS

Após validar que o tratamento de erro funciona corretamente, restaure a URL original da API para retornar a aplicação ao estado funcional.

```
// Restaurar URL correta  
private API = 'https://fakestoreapi.com/products';
```



Alterar a URL

Remova o sufixo `-erro` da URL, restaurando o endpoint correto da FakeStore API.



Salvar o arquivo

Salve o arquivo `produtos.service.ts` após a alteração.



Reiniciar o servidor

Execute `ng serve` novamente e verifique que os produtos carregam normalmente.

Resultado Após a Correção

The screenshot shows a web browser window with the address bar at `http://localhost:4200/produtos`. The page title is "Minha Loja". The navigation bar includes links for [Home](#), [Produtos](#), and [Carrinho](#), along with [Login](#) and [Logout](#) buttons. The status is "Status: Deslogado".

The main content area is titled "Produtos:" and displays "Total de produtos: 20".

The developer console is open, showing the following log entries:

- REQUEST `https://fakestoreapi.com/products` (Source: `http.interceptor.ts:6:11`)
- RESPONSE: `Object { type: 0 }` (Source: `http.interceptor.ts:20:32`)
- Lista de produtos alterada: `Array []` (Source: `lista-produtos.ts:59:15`)
- Valor total atualizado: `0` (Source: `lista-produtos.ts:63:15`)
- RESPONSE: `Object { headers: {...}, status: 200, statusText: "OK", url: "https://fakestoreapi.com/products", ok: true, type: 4, redirected: false, responseType: "cors", body: (20) [...] }` (Source: `http.interceptor.ts:20:32`)
- Lista de produtos alterada: `Array(20) [ {...}, {...}, {...}, {...}, {...}, {...}, {...}, {...}, {...}, {...}, ... ]` (Source: `lista-produtos.ts:59:15`)
- Valor total atualizado: `3240.9199999999997` (Source: `lista-produtos.ts:63:15`)

The console also shows a "Filtrar saída" (Filter output) button and tabs for "Erros", "Warnings", "Info", "Logs", and "Debug".

Resultado Final da Implementação

Com as alterações desta aula, o projeto de e-commerce agora possui um tratamento de erros completo e profissional na camada de UI.

Estados da UI

Loading, Erro e Sucesso — todos controlados e mutuamente exclusivos.

Botão de Retry

O usuário pode tentar novamente sem recarregar a página.

UI Consistente

Nenhum estado conflitante. A interface sempre reflete a realidade da operação.

Loading controlado

Indicador ativo durante a requisição, desativado ao finalizar (sucesso ou erro).

Erro visível ao usuário

Mensagem clara e amigável exibida quando a requisição falha.

Botão de retry

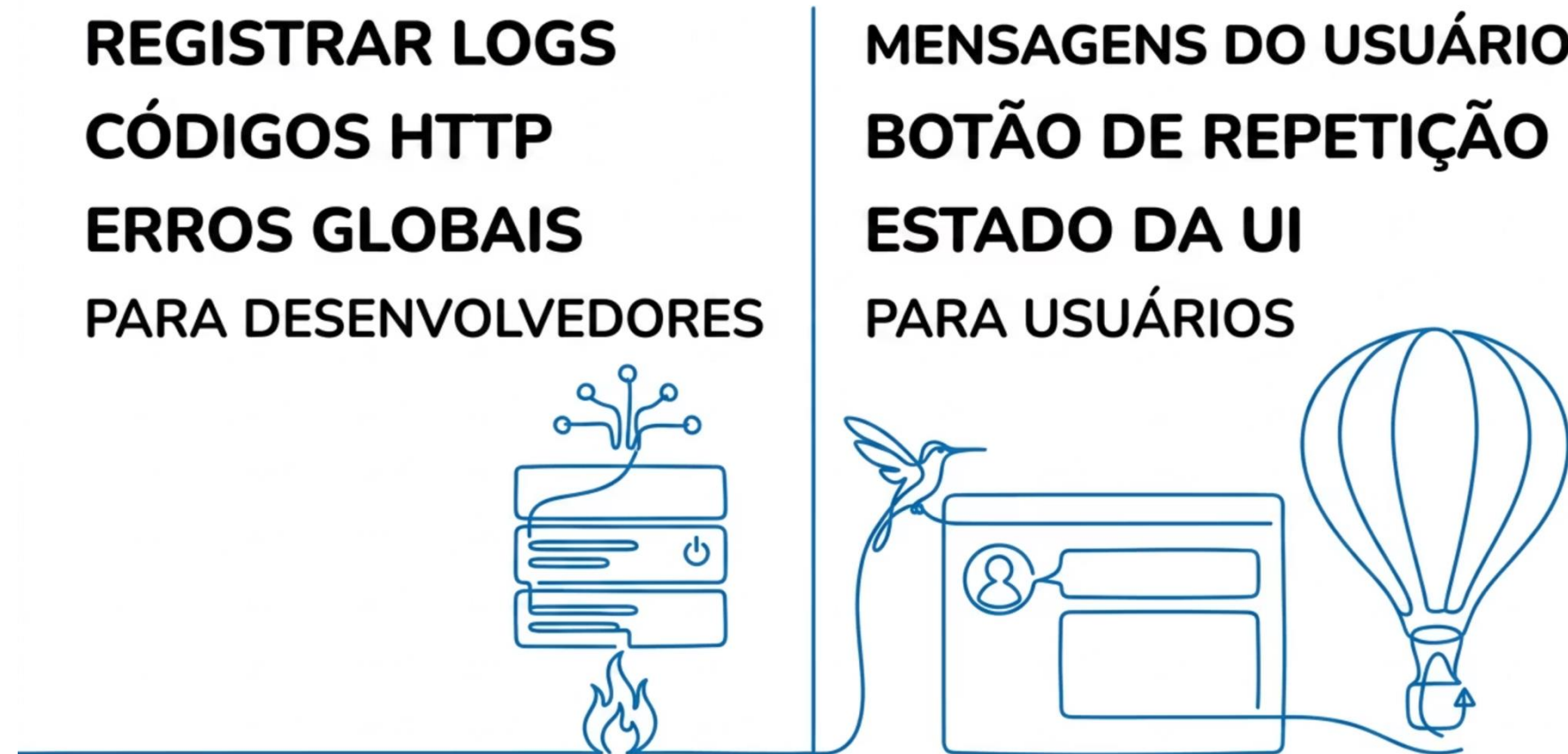
Ação de recuperação disponível sem necessidade de recarregar a página.

UI consistente

Estados mutuamente exclusivos garantem que a interface sempre faça sentido.

Arquitetura Correta — Reforço

É fundamental entender a separação de responsabilidades entre o interceptor e o componente. Cada camada tem seu papel bem definido.



O interceptor usa `catchError` enquanto o componente usa `signal` erro.

- Essa separação é um princípio de **Clean Architecture**. O interceptor não deve saber nada sobre a UI, e o componente não deve se preocupar com detalhes técnicos do HTTP.

Importante: Limitação Intencional

Nesta aula, optamos por uma abordagem simples e direta para o tratamento de erros. Existem soluções mais sofisticadas que **intencionalmente não foram usadas**.

✗ Snackbar — não usado

Componente de notificação temporária do Angular Material. Será abordado em aulas futuras quando o Material for introduzido.

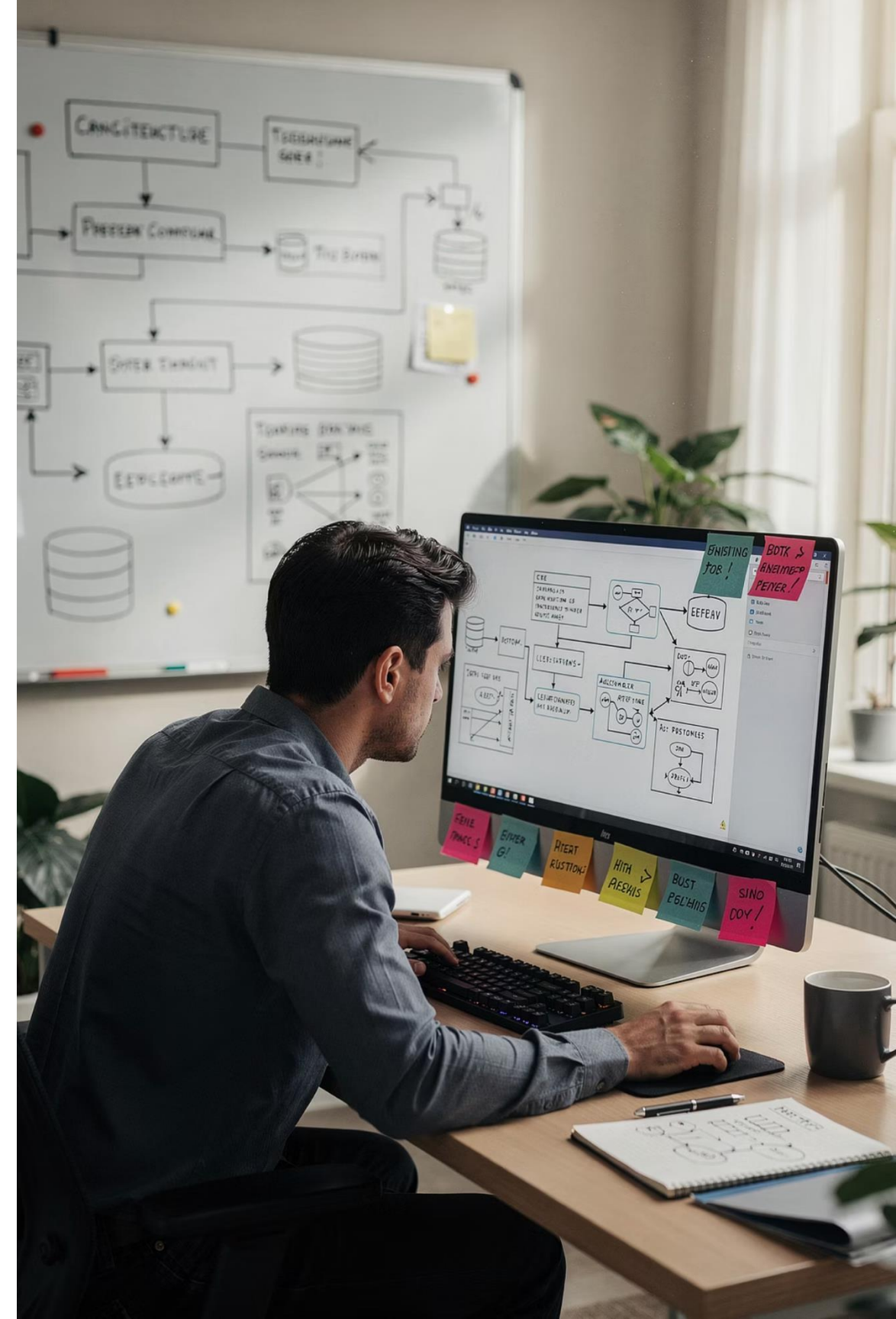
✗ Toast — não usado

Notificações flutuantes de bibliotecas como NGX-Toastr. Dependência externa desnecessária neste momento.

✗ Sistema global de mensagens — não usado

Um serviço centralizado de notificações seria mais robusto, mas adiciona complexidade antes do momento certo.

✅ **Motivo da limitação:** O foco desta aula é o entendimento do fluxo básico de estados da UI. Dominar o simples antes do complexo é a base de um bom desenvolvedor.



Conclusão desta aula:

Nesta aula evoluímos o projeto de e-commerce com um tratamento de erros profissional, focado na experiência do usuário.



Separação de responsabilidades

Interceptor para erros técnicos, componente para erros visuais.



Experiência do usuário

Mensagens claras, botão de retry e UI sempre consistente.



Estados da interface

Loading, Erro e Sucesso — mutuamente exclusivos e controlados por Signals.



Tratamento de erro na UI

Signal de erro reativo que atualiza o template automaticamente ao falhar.

Resumo das Alterações Desta Aula

Referência rápida de todos os arquivos modificados e o que foi feito em cada um.

Arquivo	Alteração	Motivo
lista-produtos.ts	Adicionado <code>erro = signal<string null>(null)</code>	Criar estado reativo de erro
lista-produtos.ts	<code>this.erro.set(null)</code> antes da requisição	Limpar erro anterior ao tentar novamente
lista-produtos.ts	Callback <code>error:</code> no subscribe	Capturar falha e atualizar signals
lista-produtos.html	Substituído bloco único por 3 blocos <code>@if</code>	Estados mutuamente exclusivos na UI
lista-produtos.html	Bloco de erro com mensagem e botão retry	Feedback visual e ação de recuperação
produtos.service.ts	URL alterada e restaurada (teste)	Simulação controlada de erro

✔ Projeto evoluído com sucesso! O e-commerce agora possui tratamento de erros completo, seguindo as melhores práticas do Angular Moderno com Standalone Components e Signals.

Dúvidas

?

